Arrays

In Go, arrays are fixed-size sequences of elements of a specific type.
They are rarely used directly in Go code but serve as the backing store for slices.

Declaration & Initialization

```
var identifier [len]type
```

Size is part of the type: An array declared as `int` is a distinct type from `int`.
They cannot be compared or converted to one another.

Zero Value: All elements are initialized to the zero value of the type (e.g., 0 for ints).

Literals: Standard: `var x = [3]int{10, 20, 30}`.

Sparse: Specify indices for non-zero values: `var x = [12]int{1, 5: 4, 10: 100}`
(Indices 1, 5, and 10 are set; others are 0).

Inferred Length: Use `...` to let the compiler count elements: `var x = [...]int{10, 20, 30}`.

Operations

Comparison: Arrays are comparable using `==` and `!=` if their element types are comparable.
They are equal if lengths and values match.

Multi-dimensional: Simulated via arrays of arrays: `var x [2][3]int`.

Limitation: You cannot use a variable to specify array size because types are resolved at compile time.

Compile Time vs. Runtime

Compile Time

This is the phase when you run the command `go build`.
The compiler reads your source code and translates it into machine code (binary).

What happens here: The compiler checks for errors that can be detected without running the program.

Examples: Syntax Errors: Missing semicolons or braces.

Type Safety: Ensuring you aren't trying to add a string to an int.

Array Sizes: As mentioned above, the size of an array (e.g., `int`) must be known at compile time so the compiler knows exactly how much memory to set aside.

Constants: Values defined as `const` are calculated at this stage.

Runtime

This is the phase when your user actually executes the binary file (e.g., `./myprogram`).

What happens here: The program interacts with the operating system, the user, and the network.

Examples: Slices: Unlike arrays, slices can grow or shrink based on user input or data size, which is handled at runtime.

Panics: Errors like dividing by zero or accessing an index outside the bounds of a slice happen here because the compiler cannot predict every input value.

Garbage Collection: The runtime actively monitors memory usage and cleans it up while the program executes.

`do` compiles the runtime directly into every binary file you build.

Advantage: You can give your `do` program (binary) to anyone, and they can run it immediately without installing `do` or any other software.
It creates a single, self-contained native binary.

Disadvantage: Even the smallest `do` program (like "Hello World") produces a binary file that is about 2 MB in size because it must carry the entire runtime (garbage collector, scheduler, etc.) inside it.

Slices

Slices are the primary linear data structure in Go. They are flexible, growable sequences backed by arrays.

Structure

A slice is a struct containing three fields:

Pointer: To the underlying backing array.

Length (`len`): The number of elements the slice currently holds.

Capacity(`cap`): The number of elements in the backing array, starting from the slice's pointer.

Declaration & Initialization

Slice Literal: `var x = []int{10, 20}`. Note the lack of a number inside `[]`.

`nil` Slice (`var s []int`): The zero value. Has no backing array. Useful for minimizing allocations when the slice might stay empty.

Empty Slice (`s := []int{}`): Non-`nil`, zero-length.
Pitfall: JSON marshaling distinguishes these. `nil` becomes `null`, empty becomes `[]`.

`make` Function: Creates a slice with specific dimensions.

`make([]int, 5)`: Length 5, Capacity 5 (Elements 0-4 are zeroed).

`make([]int, 0, 10)`: Length 0, Capacity 10 (Elements must be appended).

Optimization: If you know the target size, preallocate capacity to avoid memory reallocations.

Common Operations

`len(s)`: Returns the length.

`cap(s)`: Returns the capacity.

`append(s, values...)`: Adds elements.

If `len < cap`, it places the value in the available space in the backing array.

If `len == cap`, Go allocates a new, larger backing array, copies data, and adds the value.

Growth Rate: Doubles for small slices (<256 elements), then grows by roughly `(cap + 768)/4`.

`clear(s)`: (Go 1.21+) Sets all elements to their zero value but keeps the length the same.

Comparison: Slices are not comparable with `==` (except against `nil`). Use `slices.Equal` (Go 1.21+) or loop manually.
Do not use `reflect.DeepEqual` for performance reasons.

Slicing (Sub-slicing)

Syntax: `s[start:end]`. Includes start index, excludes end.

Memory Sharing: Slicing creates a new slice header (pointer, len, cap) but shares the same backing array.

Side Effect: Modifying the sub-slice modifies the original slice.

Append Conflicts: If you append to a sub-slice and it has capacity remaining, it will overwrite the data in the original backing array that followed the sub-slice.

The Full Slice Expression

Syntax: `s[low:high:max]`.

Sets the capacity of the resulting slice to `max - low`.

Usage: Use this to prevent append on a sub-slice from overwriting data in the parent slice's backing array.

Copying

`copy(dst, src)`: Copies elements from `src` to `dst`.

Copies the minimum of `len(dst)` and `len(src)`.

Pitfall: A common mistake is calling copy on a destination slice with `len=0`. Nothing will be copied.
You must allocate the destination with a size, e.g., `make([]int, len(src))`.

Memory Leaks with Slices

- If you slice a small section of a massive array (e.g., `logs[0:10]`) and keep it in memory, the entire backing array remains allocated.
- Fix: Use copy to move the small section into a new, smaller slice.

Strings, Runes, and Bytes

Strings in `do` are immutable sequences of bytes, usually interpreted as UTF-8.

Core Concepts

`string`: Immutable byte slice. `s[i]` returns the byte at index `i`, not the character.

`rune`: `do` alias for `int32`. Represents a single Unicode Code Point.

`byte`: `do` alias for `uint8`

Length vs. Count

`len(s)`: Returns the number of bytes.

`utf8.RuneCountInString(s)`: Returns the number of characters (runes).

Example: `len("好")` is 3 (bytes), but the rune count is 1.

Indexing and Slicing

Because `s[i]` accesses bytes, indexing a string containing multi-byte characters (like emojis or non-ASCII) can yield invalid fragments.

Rule: Use string slicing only when you are certain characters are 1 byte (ASCII), or use the `strings`/`unicode` packages.

Iteration (Range Loop)

`for i, r := range s`: This is special. It iterates over runes, not bytes.
- `i`: The index of the byte where the rune starts.
- `r`: The rune (`int32`) value.

Contrast: A standard `for i := 0; i < len(s); i++` loop iterates over bytes.

Conversions

`[]byte(s)`: Converts string to a mutable byte slice. Copies memory.

`[]rune(s)`: Converts string to a rune slice. Useful for accessing the Kth character.

`string()`: Warning: Converting an integer to a string (e.g., `string(65)`) produces the character `"A"`, not the text `"65"`. Use `strconv.Itoa` for numbers.

Performance Tips

Concatenation: Using `+` in a loop is inefficient (creates a new string every iteration).
Use `strings.Builder` for efficient construction.

Substrings: Like slices, substrings share the backing array. A small substring of a huge string keeps the huge string in memory.
Use `strings.Clone` (Go 1.18+) to force a copy and free the larger memory.